

第八讲：延续值就是你所需要的一切

异质性主体宏观经济学的计算方法

孙振越 (Jeffrey Sun)

2026 年 5 月 20 日

多伦多大学

神经值函数迭代 (nVFI)

神经值函数迭代 (nVFI)

大致而言：

1. 用带参数 θ 的神经网络对 $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ 进行参数化
2. (以某种方式) 从 Λ^{start} 中采样
3. 对每个 Λ^{start} 计算**外层贝尔曼残差** (定义见下)
4. 更新 θ , 并从第 2 步开始重复, 直至收敛

外层贝尔曼残差

考虑 Krusell–Smith 模型。

设 $\hat{V}_{\theta}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ 是一个由 θ 参数化的神经网络。

非正式地说，我们希望尽可能近似地满足，

$$\hat{V}_{\theta}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) - \mathbb{E}_{Z'} \left[\hat{V}^{\text{start}}(x^{\text{end}}; \Lambda^{\text{start}'}, Z') \mid Z \right] = 0,$$

其中 $\hat{V}^{\text{start}}(x^{\text{end}}; \Lambda^{\text{start}'}, Z')$ 是在给定 $\hat{V}_{\theta}^{\text{end}}, Z'$ 的条件下，通过**前瞻（lookahead）**计算得到的。

约定与时序

约定

1. 家庭状态在某一期的期末与下一期的期初之间保持不变：

$$x_{it}^{\text{end}} = x_{i,t+1}^{\text{start}} \quad \text{以及} \quad \Lambda_{it}^{\text{end}} = \Lambda_{i,t+1}^{\text{start}}.$$

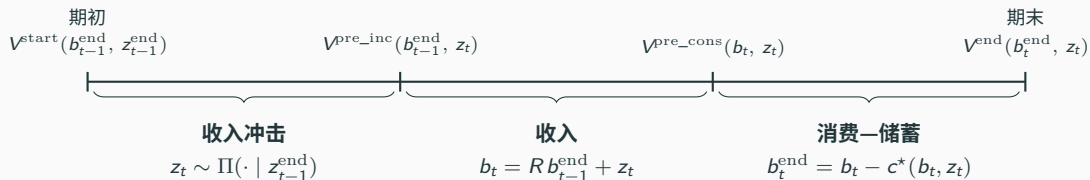
2. 总量冲击发生在家庭各期**之间**,
 - 即发生在 $v^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ 与 $v^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}'}, Z')$ 之间
3. 仅依赖于**特异性**状态的值函数**切片**或**猜测**用波浪号表示,

$$\tilde{v}^{\text{start}}(x^{\text{start}}), \quad \text{而} \quad v^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}}, Z).$$

- 即诸如 $\tilde{v}^{\text{start}}$ 这样的符号表示**取值为函数的变量**。更多细节见下。

旧版期内时序：稳态

回顾稳态版本中期内时序的结构：



或者，简化为，



新版期内时序：全局解

在引入总量冲击后，我们改写为：

$$\begin{array}{ccc}
 \tilde{V}^{\text{start}}(x^{\text{start}}) & & \tilde{V}^{\text{end}}(x^{\text{end}}) \\
 \equiv V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}}, Z) & & \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)
 \end{array}$$


家庭期

在前几讲中，我们学习了如何实现映射，

$$(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z) \mapsto (\tilde{V}^{\text{start}}, \Lambda^{\text{end}}).$$

我们称之为 $\Phi(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z) = (\tilde{V}^{\text{start}}, \Lambda^{\text{end}}).$

具体理解 Φ

这相当抽象，但请记住，如果我们将家庭状态离散化，那么，

$$\Phi\left(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z\right)=\left(\tilde{V}^{\text{start}}, \Lambda^{\text{end}}\right),$$

无非是一个从**两个矩阵和一个标量**到**两个矩阵**的函数。

它内部可能包含一个用于市场出清的循环，但本质上它以**两个矩阵和一个标量**为输入，输出**两个矩阵**。

约定：对 Λ^{start} 的依赖

- 在家庭期内，模型是确定性的。
- 因此， Λ^{end} 由 Λ^{start} 唯一确定。
- 因此，写成 $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ 在理论上是合理的。

注意：此处我尚未对如何在计算机上计算或表示 V^{end} 作出任何陈述！

这纯粹是在说 $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ 是良定义的。

约定： V^{end} 的偏应用

- 固定 Λ^{start} 和 Z ，我们可以构造，

$$\tilde{V}^{\text{end}}(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

- 用计算机科学的术语来说，这是 V^{end} 在 $(\Lambda^{\text{start}}, Z)$ 处的**偏应用**（partial application）。
- 我们将其写成一个把**函数**映为**函数**的算子

$$\mathcal{V}(\Lambda^{\text{start}}, Z) = \tilde{V}^{\text{end}}$$

- 看似抽象，但对我们而言 $\mathcal{V}$ 无非是一个从一个**矩阵**和一个**标量**到一个**矩阵**的函数。
- 预告： $\mathcal{V}$ 正是我们将尝试用**神经网络**来近似的对象。

延续值算子

- CV 是**延续值算子**，

$$\mathcal{V}(\Lambda^{\text{start}}, Z) = \tilde{V}^{\text{end}} \equiv V^{\text{end}}(\cdot; \Lambda^{\text{start}}, Z)$$

- 它返回一个**函数**。也就是说，我们可以写，

$$\mathcal{V}(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

我的观点是，**延续值算子** $\mathcal{V}$ 是值函数向带总量冲击的异质性主体（HA）模型的自然推广。

延续值就是你所需要的一切

延续值就是你所需要的一切

假设我们有一个模型：期内求解器 Φ 、延续值算子 $\mathcal{V}$ 、总量冲击过程 Π ，以及初始总量状态 $(\Lambda^{\text{start}}, Z)$ 。

那么我们就可以向前模拟该模型：

1. $\tilde{V}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z)$ (投射延续值)
2. $(\tilde{V}^{\text{start}}, \Lambda^{\text{end}}) = \Phi(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z)$ (模拟至期末)
3. 抽取 $Z' \sim \Pi(\cdot | Z)$ (抽取总量冲击)
4. $(\Lambda^{\text{start}'}, Z') \equiv (\Lambda^{\text{end}}, Z')$ (构造下一期期初状态)

给定 $\mathcal{V}$ 时的前瞻

给定 $\mathcal{V}$ ，我们还可以执行**前瞻 (lookahead)**。

1. $\tilde{V}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z)$ (投射延续值)
2. $(\tilde{V}^{\text{start}}, \Lambda^{\text{end}}) = \Phi(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z)$ (模拟至期末)
3. 抽取 $Z' \sim \Pi(\cdot \mid Z)$ (抽取总量冲击)
4. $(\Lambda^{\text{start}'}, Z') \equiv (\Lambda^{\text{end}}, Z')$ (构造下一期期初状态)
5. $\tilde{V}^{\text{end}'} = \mathcal{V}(\Lambda^{\text{start}'}, Z')$ (投射下一期期末值)
6. $(\tilde{V}^{\text{start}'}, \Lambda^{\text{end}'}) = \Phi(\tilde{V}^{\text{end}'}, \Lambda^{\text{start}'}, Z')$ (向后迭代回下一期期初值)
7. $\tilde{V}^{\text{end, 隐含}} = \mathbb{E}_{Z'}[\tilde{V}^{\text{start}'} \mid Z]$ ($\tilde{V}^{\text{end}}$ 应当等于此值)

我们将用它来推广值函数迭代 (VFI)。

旧版：典型的贝尔曼方程与算子

典型的贝尔曼方程通过下述关系定义 V ,

$$V(b, z) = \max_{c \in [0, b]} u(c) + \beta \mathbb{E}[V(b', z') \mid z]$$

这可以视为贝尔曼算子 $\mathcal{T}$ 的不动点, 该算子施行**前瞻 (lookahead)**,

$$\mathcal{T}v(b, z) \equiv \max_{c \in [0, b]} u(c) + \beta \mathbb{E}[v(b', z') \mid z]$$

于是贝尔曼方程为,

$$V = \mathcal{T}V.$$

我定义一个类似的外层贝尔曼方程。

新版：外层贝尔曼方程

真实的 $\mathcal{V}$ 也满足一个递归方程，即**外层贝尔曼方程**：

$$V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) = \mathbb{E}_{Z'} \left[\tilde{V}^{\text{start}'}(x^{\text{end}}) \mid Z \right],$$

其中

$$\tilde{V}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z)$$

$$\Lambda^{\text{end}} = \Phi_{\Lambda}(\Lambda^{\text{start}}, \tilde{V}^{\text{end}}, Z)$$

$$Z' \sim \Pi_Z(\cdot \mid Z)$$

$$\tilde{V}^{\text{end}'} = \mathcal{V}(\Lambda^{\text{end}}, Z')$$

$$\tilde{V}^{\text{start}'} = \Phi_V(\Lambda^{\text{end}}, \tilde{V}^{\text{end}'}, Z').$$

外层贝尔曼算子 $\mathcal{L}$

对任意参数化猜测 $\mathcal{V}_\theta$ ，我们可将**外层贝尔曼算子** $\mathcal{L}$ 定义为：

$$\mathcal{L}\mathcal{V}_\theta(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv \mathbb{E}_{Z'} \left[\tilde{\mathcal{V}}^{\text{start}'}(x^{\text{end}}) \mid Z \right],$$

其中，

$$\tilde{\mathcal{V}}^{\text{end}} = \mathcal{V}_\theta(\Lambda^{\text{start}}, Z)$$

$$\Lambda^{\text{end}} = \Phi_\Lambda(\Lambda^{\text{start}}, \tilde{\mathcal{V}}^{\text{end}}, Z)$$

$$Z' \sim \Pi_Z(\cdot \mid Z)$$

$$\tilde{\mathcal{V}}^{\text{end}'} = \mathcal{V}_\theta(\Lambda^{\text{end}}, Z')$$

$$\tilde{\mathcal{V}}^{\text{start}'} = \Phi_V(\Lambda^{\text{end}}, \tilde{\mathcal{V}}^{\text{end}'}, Z').$$

外层贝尔曼残差

真实的**延续值算子** $\mathcal{V}$ 由下式定义，

$$\mathcal{L}\mathcal{V} = \mathcal{V}.$$

对任意 $\mathcal{V}_\theta$ ，我们可将**外层贝尔曼残差**定义为，

$$\mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta,$$

从而可将用 $\mathcal{V}_\theta$ 近似 $\mathcal{V}$ 的问题表述为，

$$\min_{\theta} \|\mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta\|^2.$$

实现 CUIAYN

训练循环

1. 在给定 ν_θ 的条件下向前模拟，从遍历分布中采样 $(\Lambda^{\text{start}}, Z)$ 。
2. 在每个样本 $(\Lambda_i^{\text{start}}, Z_i)$ 处计算 $\mathcal{L} \nu_\theta$ ：
 - 2.1 网络在 $(\Lambda_i^{\text{start}}, Z_i)$ 处预测 $\hat{V}_\theta^{\text{end}}$ 。
 - 2.2 用 Φ 将 Λ_i^{start} 向前模拟至 Λ_i^{end} 。
 - 2.3 对每个 Z' ，用 ν_θ 在 $(\Lambda_i^{\text{end}}, Z')$ 处预测 $\hat{V}_\theta^{\text{end}'}$ 。
 - 2.4 用 Φ 向后迭代得到 $V^{\text{start}'}$ 。
 - 2.5 取期望 $\hat{V}_i^{\text{end}} \leftarrow \mathbb{E}_{Z'} [V^{\text{start}'} | Z]$ 。
3. 计算损失的梯度，

$$\frac{\partial}{\partial \theta} \sum_i \left\| \nu_\theta(\Lambda_i^{\text{start}}, Z_i) - \hat{V}_i^{\text{end}} \right\|^2,$$

将 $\hat{V}_i^{\text{end}}$ 视为固定标签，并更新 θ 。

Φ：已经完成

正如前几个笔记本中所介绍的，函数，

$$\Phi\left(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z\right) = \left(\tilde{V}^{\text{start}}, \Lambda^{\text{end}}\right),$$

可以用三个 HouseholdStage 的链来实现，

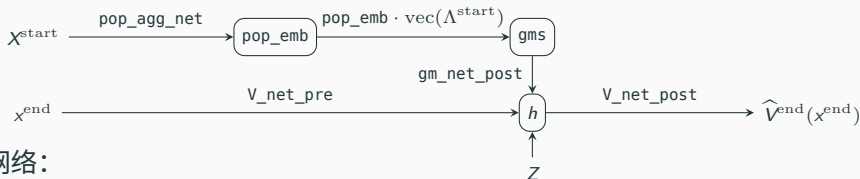
MarkovStage ◦ WealthChange ◦ ConsumptionSavings

作用于表示离散化特异性 (b, z) 网格的矩阵之上：

- 向后迭代得到 $\tilde{V}^{\text{start}}$
- 向前迭代得到 Λ^{end} 。

神经网络架构

我们采用 (Han-Yang-E 2025) 的方法压缩 Λ^{start} ，然后与家庭层面的特征相结合：



四个子网络：

1. $\text{pop_agg_net} : (b, z) \rightarrow \mathbb{R}^k$ ——逐格嵌入，对 Λ^{start} 加权聚合，得到全局矩 $\text{gms} = \sum_{(b,z)} \Lambda(b, z) \cdot \text{pop_agg_net}(b, z) \in \mathbb{R}^k$ 。
2. $\text{V_pre} : (b, z) \rightarrow \mathbb{R}^m$ ——为值函数头部提供的格级特征。
3. $\text{gm_post} : \mathbb{R}^k \rightarrow \mathbb{R}^m$ ——将 gms 投射到值函数头部的中间空间；逐格加到 V_pre 上。
4. $\text{V_post} : \mathbb{R}^{m+N_z} \rightarrow \mathbb{R}$ ——最终头部；接收加总后的特征以及 z 的独热编码，输出 $\hat{V}^{\text{end}}$ 。

代码中的神经网络

```
struct VNet{A, P, G, V}
    pop_agg_net :: A      # (b, z) → gm_dim      为 GM 提供的逐格嵌入
    V_pre        :: P      # (b, z) → mid        为 V 头部提供的逐格特征
    gm_post       :: G      # gm_dim → mid        将 GM 投射到中间空间
    V_post        :: V      # mid + N_Z → 1        组合 + Z, 读出 V_end
end

function (net::VNet)(Λ::AbstractMatrix, Z_idx::Int, p::KSPParams)
    pop_emb = net.pop_agg_net(HH_FEAT)           # (gm_dim, N_CELLS)
    gm       = pop_emb * Float32.(vec(Λ))         # (gm_dim,)    人口加权平均
    h_pre    = net.V_pre(HH_FEAT)                 # (mid, N_CELLS)
    h_gm     = net.gm_post(gm)                     # (mid,)
    h        = h_pre .+ h_gm                       # 广播 → (mid, N_CELLS)
    pred     = net.V_post(vcat(h, Z_ROWS[Z_idx])) # (1, N_CELLS)
    return reshape(pred, p.N_b, length(p.z_grid))
end
```


前瞻算子 $\mathcal{L}V_\theta$

```
function lookahead(hh, vnet::VNet,  $\Lambda$ _start::AbstractMatrix, Z_idx::Int, p::KSParams)
    V_end_now = Float64.(vnet( $\Lambda$ _start, Z_idx, p))
    env_now = make_env(hh; ks_prices(integrate_K(hh,  $\Lambda$ _start), p.Z_vals[Z_idx], p)...)
    backward!(hh, V_end_now, env_now) # 确定储蓄策略
     $\Lambda$ _end = forward!(hh,  $\Lambda$ _start)

    V_end_label = zeros(Float64, p.N_b, length(p.z_grid))
    for Z_next in 1:N_Z
        V_start_next = inner_solve_backward(hh, vnet,  $\Lambda$ _end, Z_next, p)
        V_end_label .+= p. $\Pi_Z$ [Z_idx, Z_next] .* V_start_next
    end
    return (; V_end_label,  $\Lambda$ _end)
end
```

训练损失

将 $\text{target} \equiv \hat{V}_i^{\text{end}}$ 视为固定标签，对 `vnet` 进行自动微分。

```
function build_targets(hh, vnet::VNet, batch, p::KSParams)
    targets = NamedTuple{(: $\Lambda$ , :Z_idx, :target), Tuple{Matrix{Float64}, Int, Matrix{Float32}}{}}[]
    for ( $\Lambda$ _start, Z_idx) in batch
        out = lookahead(hh, vnet,  $\Lambda$ _start, Z_idx, p)
        push!(targets, (;  $\Lambda$ =copy( $\Lambda$ _start), Z_idx, target=Float32.(out.V_end_label)))
    end
    return targets
end

function bellman_residual_loss(vnet::VNet, targets, p::KSParams)
    L = 0f0
    for ( $\Lambda$ , Z_idx, target) in targets
        pred = vnet( $\Lambda$ , Z_idx, p)
        scale = max(var(target), 1f-6) # 按目标尺度归一化
        L += sum((pred .- target).^2) / (length(pred) * scale)
    end
    return L / length(targets)
end
```

对 Λ^{start} 采样

MCMC 风格：在给定当前 V_θ 的条件下进行模拟，预烧 (burn in)，然后子采样。

```
function sim_one_step(hh, vnet::VNet,  $\Lambda$ ::AbstractMatrix, Z_idx::Int, p::KSParams)
    V_end = Float64.(vnet( $\Lambda$ , Z_idx, p))
    env = make_env(hh; ks_prices(integrate_K(hh,  $\Lambda$ ), p.Z_vals[Z_idx], p)...)
    backward!(hh, V_end, env)
    return forward!(hh,  $\Lambda$ )
end

function sample_ergodic(hh, vnet::VNet, p::KSParams; T=200, burn=100, every=2, rng=MersenneTwister(8421))
     $\Lambda$  = copy( $\Lambda_{ss}$ )
    Z_idx = 1
    batch = Tuple{Matrix{Float64}, Int}[]
    for t in 1:T
         $\Lambda$  = sim_one_step(hh, vnet,  $\Lambda$ , Z_idx, p)
        Z_idx = sample_Z(Z_idx, p. $\Pi_Z$ , rng)
        if t > burn && (t - burn) % every == 0
            push!(batch, (copy( $\Lambda$ ), Z_idx))
        end
    end
    return batch
end
```

训练循环

```
function train!(vnet::VNet, hh, p::KSPParams; epochs=50, lr=1e-3, T=200, burn=100, every=4)
    opt = Flux.setup(Adam(lr), vnet)
    hist = Float64[]
    rng = MersenneTwister(1729)
    for epoch in 1:epochs
        batch = sample_ergodic(hh, vnet, p; T, burn, every, rng=MersenneTwister(rand(rng, UInt32)))
        targets = build_targets(hh, vnet, batch, p)
        loss, grads = withgradient(vnet) do net
            bellman_residual_loss(net, targets, p)
        end
        Flux.update!(opt, vnet, grads[1])
        push!(hist, Float64(loss))
    end
    return hist
end
```

初始化与预训练

在扰动后的 Λ_{ss} 和所有 Z 上，对 V_{ss} 进行预训练。

```
function perturb_Λ(Λ::AbstractMatrix, σ, rng)
    Λp = Λ .* (1 .+ σ .* randn(rng, size(Λ)))
    Λp = max.(Λp, 0)
    return Λp ./ sum(Λp)
end

function pretrain_to_ss!(vnet::VNet, V_ss_target, p::KSPParams; epochs=300, lr=5e-3, n_Λ=6, σ=0.10, seed=4747)
    opt = Flux.setup(Adam(lr), vnet)
    target = Float32.(V_ss_target)
    rng = MersenneTwister(seed)
    hist = Float64[]
    for epoch in 1:epochs
        Λ_cloud = [perturb_Λ(Λ_ss, σ, rng) for _ in 1:n_Λ]
        loss, grads = withgradient(vnet) do net
            L = 0f0
            for Λ in Λ_cloud, Z_idx in 1:N_Z
                pred = net(Λ, Z_idx, p)
                L += sum((pred .- target).^2)
            end
            L / (n_Λ * N_Z * length(target))
        end
    end
```

推广

推广：模型

- 对任何能够实现 Φ 的模型，这一通用方法都是良定义的。
- 如果你无法实现 Φ ，那么问题并不在于总量不确定性。
- 我提供了 `HouseholdStages.jl`，用于构建高性能的家庭模块。
- 可以处理：空间、多部门、世代交叠（OLG）、搜寻—匹配、新凯恩斯、带摩擦的耐用资产（如住房）、理性疏忽、Epstein–Zin 偏好

推广：分割市场

分割市场模型，例如包含多个区位或多个部门的模型，带来了额外的挑战。

我提出：

- **分割广义矩**，将 Han–Yang–E (2025) 推广为以模型一致的方式同时压缩分段（如区位）数据与总量数据。
- **轻量网络头部**方法，将含 n 个分段、 m 个家庭状态的分割市场模型的训练，从 $O(nm)$ 加速到实质上的 $O(n + m)$

推广：方法

原则上，nVFI 算法并不依赖于以下这些实现细节：

- $\mathcal{V}_\theta$ 的参数化：我使用神经网络，但该方法只需要某种可训练的函数近似器。
- 值函数的表示：我使用在离散化特异性状态上的矩阵，但你也可以使用样条、Chebyshev 多项式，甚至另一个神经网络。
- 人口的表示：我使用在离散化特异性状态上的矩阵，但你也可以使用矩、有限主体等。
- 期内求解器：我主要使用基于网格的方法，但你也可以使用 EGM、Reiter 等。

只要你能实现**期内求解器** Φ ，该算法就是良定义的。

与 HouseholdStages.jl 的关系

本方法与我在 Household Stages 上的工作之间存在如下深层联系：

- 离散时间的异质性主体（HA）模型具有基于**函数复合**的**复合结构**
- 相比之下，连续时间的 HA 模型具有基于**可加复合**的复合结构。

关于 Household Stages 的论文与软件包探讨了这一点对**期内求解器** Φ 的意涵
受此启发，nVFI 用 $\mathcal{V}$ 推广了贝尔曼方程，并将其用于全局求解。